Transformación de Fechas y horas

```
#Importamos librerías
from datetime import datetime, date, time, timedelta
import datetime
import pandas as pd
import numpy as np
```

```
#AÑO
ahora = datetime.datetime.now()
print("La fecha y hora actual es: ", ahora)

#Este año
anio = ahora.year
print("El año es: ", anio)

#Año proximo
año_proximo = anio + 1
print("El año próximo es: ", año_proximo)
```

```
La fecha y hora actual es:  2026-04-21 22:27:58.493586
El año es:  2026
El año próximo es:  2027
```

```
#Mes

#Mes actual
mes_actual = ahora.month
print("El mes actual es: ", mes_actual)

#Mes próximo
mes_proximo = mes_actual + 1
print("El mes próximo es: ", mes_proximo)

#Cambio el tipo de dato
print(type(mes_proximo))
print(type(str(mes_proximo)))
```

```
El mes actual es:  4
El mes próximo es:  5
<class 'int'>
<class 'str'>
```

```
#DIA

#Dia actual
dia_actual = ahora.day
print("El día actual es: ", dia_actual)

#Dia próximo
dia_proximo = dia_actual + 1
print("El día próximo es: ", dia_proximo)
```

```
El día actual es:  21
El día próximo es:  22
```

```
#HORA

#Hora GMT
ahora = datetime.datetime.now()
print("La hora Greenwich Mean Time (GMT) es: ", ahora)

#Hora Argentina (Le restamos 3 horas)
hora_arg = ahora - timedelta(hours=3)
print("La hora en Argentina es:", hora_arg)
```

```
La hora Greenwich Mean Time (GMT) es:  2026-04-21 22:31:35.880890
La hora en Argentina es: 2026-04-21 19:31:35.880890
```

```
# Formatear la hora
hora_formateada = ahora.strftime("%Y-%m-%d %H:%M:%S")

# Imprimir la hora formateada
print("La hora actual es:", hora_formateada)
```

```
La hora actual es: 2026-04-21 22:31:35
```

Transformación de Strings a enteros

```
#Cambio el tipo de dato, de string a enteros

numero = "15"

print(type(numero))

print(f"El numero ingresado es : {numero} y el tipo es:  {type(numero)} ")
```

```
<class 'str'>
El numero ingresado es : 15 y el tipo es:  <class 'str'>
```

```
#convertimos el dato de string a enteros
numero = int(numero)

print(f"El numero ingresado es : {numero} y el tipo es:  {type(numero)} ")
```

```
El numero ingresado es : 15 y el tipo es:  <class 'int'>
```

Transformación de Strings a decimales

```
#Cambio el tipo de dato, de string a enteros

numero = "10.1"

print(f"El numero ingresado es : {numero} y el tipo es:  {type(numero)} ")
```

```
El numero ingresado es : 10.1 y el tipo es:  <class 'str'>
```

```
#convertimos el dato de estring a enteros
numero = float(numero)

print(f"El numero ingresado es : {numero} y el tipo es:  {type(numero)} ")
```

```
El numero ingresado es : 10.1 y el tipo es:  <class 'float'>
```

Transformación, join de datasets

```
# Notas de Matemática y Lengua
df1 = pd.DataFrame({'Alumno': ['Alumno0', 'Alumno1', 'Alumno2'],
                    'Matermática': ['5', '6', '7'],
                    'Lengua': ['7', '7', '8']
                    })

# Notas de Historia y Geografía
df2 = pd.DataFrame({'Alumno': ['Alumno0', 'Alumno2', 'Alumno3'],
                    'Historia': ['4', '5', '4'],
                    'Geografía': ['6', '7', '9']
                    })

# Mostramos los dataframes
print(df1)
print('           ')
print(df2)
```

```
    Alumno Matermática Lengua
0  Alumno0           5      7
1  Alumno1           6      7
2  Alumno2           7      8

    Alumno Historia Geografía
0  Alumno0        4         6
1  Alumno2        5         7
2  Alumno3        4         9
```

```
# Hacemos join de los dataframes y mostramos las notas
```

```
merged_df = pd.merge(df1, df2, on='Alumno', how='left')
print(merged_df)
```

```
    Alumno Matermática Lengua Historia Geografía
0  Alumno0           5      7        4         6
1  Alumno1           6      7      NaN       NaN
2  Alumno2           7      8        5         7
```

Substitución de datos faltante por un tipo de dato default

```
#Reemplazamos nulos por ceros

# Creando un DataFrame
df = pd.DataFrame({'columna1': [1, 2, None, 4],
                   'columna2': [5, None, None, 8]})

print(df)
```

```
   columna1  columna2
0       1.0       5.0
1       2.0       NaN
2       NaN       NaN
3       4.0       8.0
```

```
# Rellenando los valores nulos con 0

df = df.fillna(0) #Podemos reemplazar el cero por el valor que necesitemos
print(df)
```

```
   columna1  columna2
0       1.0       5.0
1       2.0       0.0
2       0.0       0.0
3       4.0       8.0
```

Filtrar datos que rompen reglas de negocio o porque no los necesitemos

```
# Creamos un DataFrame de ejemplo
data = {'producto': ['A', 'B', 'C', 'D'],
        'precio': [5, 15, 8, 20]}
df = pd.DataFrame(data)

print(df)
```

```
  producto  precio
0        A       5
1        B      15
2        C       8
3        D      20
```

```
# Filtrar los productos con precio menor a 10
df_filtrado = df[df['precio'] < 10]
print(df_filtrado)
```

```
  producto  precio
0        A       5
2        C       8
```

Eliminar columnas que no son necesarias

```
# Creamos un DataFrame de ejemplo
data = {'nombre': ['Ana', 'Pedro', 'María'],
        'edad': [30, 25, 32],
        'ciudad': ['Madrid', 'Barcelona', 'Sevilla'],
        'correo': ['ana@gmail.com', 'pedro@gmail.com', 'maria@gmail.com'],
        'telefono': [123456789, 987654321, 567891234]}
df = pd.DataFrame(data)

print(df)
```

```
  nombre  edad     ciudad           correo   telefono
0    Ana    30     Madrid    ana@gmail.com  123456789
1  Pedro    25  Barcelona  pedro@gmail.com  987654321
```

```
2  María    32    Sevilla  maria@gmail.com  567891234
```

```
# Eliminar las columnas 'correo' y 'telefono'
columnas_a_eliminar = ['correo', 'telefono']

#axis=1: Se refiere a las columnas.
#inplace=True: modifica el DataFrame original en lugar de crear una copia y realizar los cambios en ella
df.drop(columnas_a_eliminar, axis=1, inplace=True)

print(df)
```

```
  nombre  edad     ciudad
0    Ana    30     Madrid
1  Pedro    25  Barcelona
2  María    32    Sevilla
```

Agrupamiento

```
# Creamos un DataFrame de ventas
data = {'Vendedor': ['Ana', 'Pedro', 'Ana', 'María', 'Pedro', 'Ana'],
        'Monto': [100, 200, 150, 300, 100, 50],
        'Operaciones': [1, 1, 2, 1, 3, 1]}
df_ventas = pd.DataFrame(data)

print("Datos originales:")
print(df_ventas)
```

```
Datos originales:
  Vendedor  Monto  Operaciones
0      Ana    100            1
1    Pedro    200            1
2      Ana    150            2
3    María    300            1
4    Pedro    100            3
5      Ana     50            1
```

```
# Agrupamos por 'Vendedor' y sumamos sus montos
# El proceso es: 1. Agrupar, 2. Seleccionar columna, 3. Aplicar función (sum, mean, count)
ventas_totales = df_ventas.groupby('Vendedor')['Monto'].sum().reset_index()

print("\nVentas totales por vendedor:")
print(ventas_totales)
```

```
Ventas totales por vendedor:
  Vendedor  Monto
0      Ana    300
1    María    300
2    Pedro    300
```

Conversión de Mayúsculas

```
# Creamos un DataFrame con nombres variados
data = {'Estudiante': ['ana gomez', 'PEDRO RODRIGUEZ', 'mAría loPez']}
df_nombres = pd.DataFrame(data)

print("Datos originales:")
print(df_nombres)
```

```
Datos originales:
        Estudiante
0        ana gomez
1  PEDRO RODRIGUEZ
2      mAría loPez
```

```
# Convertir a MAYÚSCULAS
df_nombres['Estudiante_Mayus'] = df_nombres['Estudiante'].str.upper()
print(df_nombres)
```

```
        Estudiante Estudiante_Mayus
0        ana gomez        ANA GOMEZ
1  PEDRO RODRIGUEZ  PEDRO RODRIGUEZ
2      mAría loPez      MARÍA LOPEZ
```

```
# Convertir a minúsculas
df_nombres['Estudiante_Minus'] = df_nombres['Estudiante'].str.lower()
print(df_nombres)
```

```
          Estudiante Estudiante_Mayus Estudiante_Minus
0        ana gomez        ANA GOMEZ        ana gomez
1  PEDRO RODRIGUEZ  PEDRO RODRIGUEZ  pedro rodriguez
2      mAría loPez      MARÍA LOPEZ      maría lopez
```

📝 Ejercicios de Transformación de Datos

1. Transformación de Fechas y Horas: Un cargamento de datos llega siempre exactamente 7 días después de la fecha de hoy. Crea un script que obtenga la fecha actual y le sume una semana (7 días), mostrando el resultado en formato día/mes/año

Pista: Usa la librería datetime y timedelta

```
#Ejercicio 1
#AÑO
ahora = datetime.datetime.now()
print("La fecha y hora actual es: ", ahora)

semana_mas = ahora + timedelta(days=7)
print("La fecha y hora de la semana que viene es: ", semana_mas)
```

```
La fecha y hora actual es:  2026-04-21 22:54:08.713081
La fecha y hora de la semana que viene es:  2026-04-28 22:54:08.713081
```

2. Transformación de Strings a Números: Convierte la cadena "99.5" a un tipo de dato flotante y luego a un tipo de dato entero. ¿Qué sucede con el valor decimal al hacer la conversión final?

```
#Ejercicio 2

cadena_numero = "99.5"

print(f"Cadena original: {cadena_numero}, Tipo: {type(cadena_numero)}")

# Convertir a flotante
numero_flotante = float(cadena_numero)
print(f"Después de convertir a flotante: {numero_flotante}, Tipo: {type(numero_flotante)}")

# Convertir a entero
numero_entero = int(numero_flotante)
print(f"Después de convertir a entero: {numero_entero}, Tipo: {type(numero_entero)}")

print("\nAl convertir de flotante a entero, la parte decimal (0.5) se trunca, no se redondea.")
```

```
Cadena original: 99.5, Tipo: <class 'str'>
Después de convertir a flotante: 99.5, Tipo: <class 'float'>
Después de convertir a entero: 99, Tipo: <class 'int'>

Al convertir de flotante a entero, la parte decimal (0.5) se trunca, no se redondea.
```

3. Filtrado de Datos: Dado el siguiente DataFrame, filtra los datos para mostrar solo las ventas que superan los 500 y son del producto 'A'.

```
#Ejercicio 3
data = {'producto': ['A', 'B', 'A', 'C'], 'ventas': [600, 450, 750, 300]}
df = pd.DataFrame(data)
print(df)
print('                        ')
```

```
  producto  ventas
0        A     600
1        B     450
2        A     750
3        C     300
```

```
# Filtrar las ventas que superan los 500 y son del producto 'A'
df_filtrado_ejercicio3 = df[(df['ventas'] > 500) & (df['producto'] == 'A')]

print("Ventas filtradas (más de 500 y producto 'A'):")
print(df_filtrado_ejercicio3)
```

```
Ventas filtradas (más de 500 y producto 'A'):
  producto  ventas
```

```
0        A     600
2        A     750
```

4. Agrupación y Agregación de Datos: Utiliza el DataFrame anterior y agrúpalo por 'producto' para calcular la suma total de 'ventas' por cada producto.

```
#Ejercicio 4

# Agrupar por 'producto' y sumar las 'ventas'
ventas_por_producto = df.groupby('producto')['ventas'].sum().reset_index()

print("Ventas totales por producto:")
print(ventas_por_producto)
```

```
Ventas totales por producto:
  producto  ventas
0        A    1350
1        B     450
2        C     300
```

5. Unión de DataFrames: Tienes dos DataFrames, uno con datos de empleados y otro con sus salarios. Únelos para que en un solo DataFrame aparezca el nombre del empleado y su salario.

```
#Ejercicio 5
df_empleados = pd.DataFrame({'id_empleado': [101, 102, 103], 'nombre': ['Juan', 'María', 'Pedro']})
df_salarios = pd.DataFrame({'id_empleado': [102, 101, 103], 'salario': [1500000, 1600000, 1750000]})
print(df_empleados)
print('                        ')
print(df_salarios)
print('                        ')
```

```
   id_empleado nombre
0          101   Juan
1          102  María
2          103  Pedro

   id_empleado  salario
0          102  1500000
1          101  1600000
2          103  1750000
```

```
# Unir los DataFrames por 'id_empleado'
df_unido = pd.merge(df_empleados, df_salarios, on='id_empleado', how='inner')

# Seleccionar solo las columnas 'nombre' y 'salario' para el resultado final
df_resultado_ejercicio5 = df_unido[['nombre', 'salario']]

print("DataFrame unido con nombre y salario de empleados:")
print(df_resultado_ejercicio5)
```

```
DataFrame unido con nombre y salario de empleados:
  nombre  salario
0   Juan  1600000
1  María  1500000
2  Pedro  1750000
```

6. Reemplazo de Valores: En el siguiente DataFrame, reemplaza todos los valores None en la columna 'edad' por el valor de la edad promedio de la columna.

```
#Ejercicio 6

data = {'nombre': ['Ana', 'Luis', 'Sofía'], 'edad': [25, np.nan, 30]}
df = pd.DataFrame(data)
print(df)
print('                        ')
```

```
  nombre  edad
0    Ana  25.0
1   Luis   NaN
2  Sofía  30.0
```

```
# Calcular la edad promedio de la columna 'edad'
edad_promedio = df['edad'].mean()

# Reemplazar los valores None (NaN) en la columna 'edad' por la edad promedio
df['edad'] = df['edad'].fillna(edad_promedio)
```

```
print("DataFrame después de reemplazar None por la edad promedio:")
print(df)
```

```
DataFrame después de reemplazar None por la edad promedio:
  nombre  edad
0    Ana  25.0
1   Luis  27.5
2  Sofía  30.0
```

7. Eliminación de Filas: Dado el DataFrame anterior, elimina la fila que contiene el valor None

```
# Ejercicio 7
# Eliminar filas con valores nulos
df_sin_nulos = df.dropna()

print("DataFrame después de eliminar la fila con None:")
print(df_sin_nulos)
```

```
DataFrame después de eliminar la fila con None:
  nombre  edad
0    Ana  25.0
2  Sofía  30.0
```

8. Transformación de Datos de una Columna: En el siguiente DataFrame, convierte todos los valores de la columna 'ciudad' a mayúsculas.

```
#Ejercicio 8
df_ciudades = pd.DataFrame({'ciudad': ['madrid', 'barcelona', 'sevilla']})
print(df_ciudades)
print('                ')

# Convertir la columna 'ciudad' a mayúsculas
df_ciudades['ciudad'] = df_ciudades['ciudad'].str.upper()

print("DataFrame con ciudades en mayúsculas:")
print(df_ciudades)
```

```
      ciudad
0     madrid
1  barcelona
2    sevilla

DataFrame con ciudades en mayúsculas:
      ciudad
0     MADRID
1  BARCELONA
2    SEVILLA
```

```
# Convertir la columna 'ciudad' a mayúsculas
df_ciudades['ciudad'] = df_ciudades['ciudad'].str.upper()

print("DataFrame con ciudades en mayúsculas:")
print(df_ciudades)
```

```
DataFrame con ciudades en mayúsculas:
      ciudad
0     MADRID
1  BARCELONA
2    SEVILLA
```

9. Creación de una Nueva Columna: Usando el DataFrame de df_ciudades, crea una nueva columna llamada 'país' y asigna el valor 'España' a todas las filas.

```
#Ejercicio 9
df_ciudades['país'] = 'España'
print("DataFrame con la nueva columna 'país':")
print(df_ciudades)
```

```
DataFrame con la nueva columna 'país':
      ciudad    país
0     MADRID  España
1  BARCELONA  España
2    SEVILLA  España
```

10. Eliminar Duplicados: En el siguiente DataFrame, elimina las filas duplicadas.

```
#Ejercicio 10
df_duplicados = pd.DataFrame({'A': [1, 2, 2, 3], 'B': ['x', 'y', 'y', 'z']})
print(df_duplicados)
print('               ')
```

```
   A  B
0  1  x
1  2  y
2  2  y
3  3  z
```

```
# Eliminar filas duplicadas
df_sin_duplicados = df_duplicados.drop_duplicates()

print("DataFrame después de eliminar duplicados:")
print(df_sin_duplicados)
```

```
DataFrame después de eliminar duplicados:
   A  B
0  1  x
1  2  y
3  3  z
```